TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Laboratory work N5

Course: **Programming Languages Concepts**

Theme: **Procedures and Functions in PL**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. LABORATORY WORK TASK 3
2. PROGRAM CODE AND RESULTS 3
3. CONCLUSIONS 9
4. REFERENCES 10

1. LABORATORY WORK TASK

The purpose of this laboratory work is to compare the used programming language, its syntax, and its capabilities with other programming languages. Ruby was selected for this laboratory work because it is the language used in the practical tasks and it has concise syntax for methods, recursion, arrays, and numeric processing. Compared with C++, Ruby requires less boilerplate and no explicit type declarations, which makes short educational programs easier to read. Compared with Pascal-style languages, Ruby is more dynamic and expressive, but it does not provide true pass-by-reference for immutable numeric values, so reference-like behaviour must be demonstrated through a mutable container.

Implemented tasks:

1. Write procedure Maximum for three real numbers that accepts values from the keyboard.

2. Write two functions of a number raised to the square using pass-by-value and reference-style argument handling.

3. Write a recursive function to calculate Fibonacci numbers and display the values from 0 to 10.

4. Use recursion to calculate the factorial of a user-entered non-negative integer.

2. PROGRAM CODE AND RESULTS

<u>Task 1. Procedure Maximum for three real numbers</u>

```
#!/usr/bin/env ruby

$stdout.sync = true

def read_real(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = Float(input.strip, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid real number.'
  end
```

```ruby
end

def format_number(number)
  return number.to_i.to_s if number == number.to_i

  format('%.2f', number)
end

def maximum(first, second, third)
  puts "Maximum value: #{format_number([first, second, third].max)}"
end

first = read_real('Enter first real number: ')
second = read_real('Enter second real number: ')
third = read_real('Enter third real number: ')

maximum(first, second, third)
```

```
src : fish — Konsole
┌─[X]─[07:55-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└─╼_6r4ndf4th3r$./task_1_maximum_procedure.rb
Enter first real number: 23.12
Enter second real number: 4.065
Enter third real number: 6.2
Maximum value: 23.12
┌─[X]─[07:55-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└─╼_6r4ndf4th3r$
```

Task 2. Square a number by value and by reference

Ruby passes object references by value, and numeric objects are immutable. Because of that, the first method squares a copied number directly, while the second method demonstrates reference-style behaviour by modifying a one-element array that acts as a mutable holder for the number.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

def read_real(prompt)
  loop do
    print prompt
```

```
    input = gets
    exit if input.nil?

    value = Float(input.strip, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

def format_number(number)
  return number.to_i.to_s if number == number.to_i

  format('%.2f', number)
end

def square_by_value(number)
  number * number
end

def square_by_reference(number_holder)
  number_holder[0] *= number_holder[0]
end

input_number = read_real('Enter a number: ')

value_argument = input_number
value_result = square_by_value(value_argument)

reference_argument = [input_number]
square_by_reference(reference_argument)

puts "Original number: #{format_number(input_number)}"
puts "square_by_value result: #{format_number(value_result)}"
puts "Argument after square_by_value: #{format_number(value_argument)}"
puts "Argument after square_by_reference: #{format_number(reference_argument[0])}"
```

```
src : fish — Konsole
┌[X]-[07:56-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_2_square_functions.rb
Enter a number: 7
Original number: 7
square_by_value result: 49
Argument after square_by_value: 7
Argument after square_by_reference: 49
┌[X]-[07:57-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$
```

Task 3. Recursive Fibonacci function

A recursive Ruby function calculates Fibonacci numbers and is used to display the values from F(0) to F(10). The output confirms that F(10) equals 55.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

def fibonacci(index)
  return index if index < 2

  fibonacci(index - 1) + fibonacci(index - 2)
end

puts 'Fibonacci sequence from F(0) to F(10):'

(0..10).each do |index|
  puts "F(#{index}) = #{fibonacci(index)}"
end
```

```
src : fish — Konsole
┌[X]-[07:57-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_3_recursive_fibonacci.rb
Fibonacci sequence from F(0) to F(10):
F(0) = 0
F(1) = 1
F(2) = 1
F(3) = 2
F(4) = 3
F(5) = 5
F(6) = 8
F(7) = 13
F(8) = 21
F(9) = 34
F(10) = 55
┌[X]-[07:57-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$
```

Task 4. Recursive factorial function

A recursive Ruby function calculates the factorial of a non-negative integer entered by the user, including the base case 0! = 1.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

def read_non_negative_integer(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = Integer(input.strip, exception: false)
    return value if !value.nil? && value >= 0

    puts 'Please enter a non-negative whole number.'
  end
end

def factorial(number)
  return 1 if number.zero?

  number * factorial(number - 1)
```

```ruby
end

number = read_non_negative_integer('Enter a non-negative integer: ')
puts "Factorial of #{number} is #{factorial(number)}"
```

```
src : fish — Konsole
┌[X]-[07:58-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_4_recursive_factorial.rb
Enter a non-negative integer: 4
Factorial of 4 is 24
┌[X]-[07:59-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_4_recursive_factorial.rb
Enter a non-negative integer: 9
Factorial of 9 is 362880
┌[X]-[07:59-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_4_recursive_factorial.rb
Enter a non-negative integer: 15
Factorial of 15 is 1307674368000
┌[X]-[07:59-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_4_recursive_factorial.rb
Enter a non-negative integer: 0
Factorial of 0 is 1
┌[X]-[07:59-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$./task_4_recursive_factorial.rb
Enter a non-negative integer: -56
Please enter a non-negative whole number.
Enter a non-negative integer: 20
Factorial of 20 is 2432902008176640000
┌[X]-[07:59-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-5/src]
└-_6r4ndf4th3r$
```

3. CONCLUSIONS

In this laboratory work, the required tasks were written in Ruby. The maximum, squaring, Fibonacci, and factorial tasks showed how Ruby methods can be used to structure logic clearly with compact syntax.

The comparison with other programming languages is most visible in parameter passing. In C++, pass-by-value and pass-by-reference are built directly into function signatures. In Ruby, numbers are immutable and object references are passed by value, so the reference-style task needs a mutable wrapper to simulate direct modification. This difference makes Ruby concise and expressive, but also shows that its capabilities and syntax differ from lower-level languages. Overall, Ruby is well suited for small laboratory tasks and still allows meaningful comparison with other programming languages.

4. REFERENCES

1. https://docs.ruby-lang.org/en/3.4/syntax/methods_rdoc.html
2. https://docs.ruby-lang.org/en/3.4/syntax/calling_methods_rdoc.html
3. https://docs.ruby-lang.org/en/3.4/Integer.html
4. https://docs.ruby-lang.org/en/3.4/Array.html